

DOCUMENTEACIÓN
TRABAJO — CAFETERÍA MULTI-HILOS

Eder Martínez Castro

3 de diciembre de 2025

Índice general

1.	Análisis y Especificación de Requisitos	2
1.1.	Descripción del problema	2
1.2.	Requisitos funcionales	2
1.3.	Requisitos no funcionales	2
1.4.	Casos de uso	3
1.5.	Historias de usuario	3
2.	Arquitectura del sistema	4
2.1.	Arquitectura	4
2.2.	Clases	5
2.3.	Patrones de diseño	8
2.4.	Stack empleado	9
2.5.	Interfaz	9

1. Análisis y Especificación de Requisitos

1.1. Descripción del problema

Se requiere un **programa en Java** que **simule una cafetería** utilizando **hilos**, los **clientes** llegaran a la cafetería y serán atendidos por los **camareros** y los **cafés** los hara el **barista**.

- Cada **cliente** tendrá nombre y **tiempoEspera**.
- Los **camareros** reciben los **cafés** del barista y lo entregan al cliente.
- El **barista** preparará los **cafés**.
- Si el tiempo de espera del cliente se acaba, **se marchará**.
- El sistema terminará cuando **todos los clientes han recibido su cafe o se fueron**.

1.2. Requisitos funcionales

A continuación, se listan los requisitos funcionales del sistema:

ID	Descripción
RF-01	Crear clientes: nombre:String y tiempoEspera:int (ms).
RF-02	Crear camareros: instanciar n camareros que serán los consumidores, cada uno se ejecutará en un hilo independiente.
RF-03	Crear barista: instanciar un barista que será nuestro productor.
RF-04	Preparar café: el barista simulara con <code>Threads.sleep</code> el tiempo que tarda en prepararlo.
RF-05	Recoger café: el camarero recogerá el café cuando el barista ya lo haya hecho.
RF-06	Abandono por espera: si el cliente no recibe su café y se le acabo su tiempoEspera, se marchará.
RF-07	Entrega del café: cuando el camarero le entregará el café al cliente si sigue esperando.

1.3. Requisitos no funcionales

A continuación, se listan los requisitos no funcionales del sistema:

ID	Descripción
RNF-01	Lenguaje: El programa se realizará en Java y JavaFX.
RNF-02	Método a emplear: Se utilizará el método <code>Productor-Consumidor</code> .
RNF-03	Observabilidad: Los mensjes mostrados deben ser claros y simples.
RNF-04	Cierre ordenado: todos los hilos terminaran de forma correcta antes de que termine el programa.

1.4. Casos de uso

Descripción de casos de uso principales:

Caso de uso	Descripción
CU-01 Llegada y pedido	El cliente llega, y lo atiende el camarero y esperar su café hasta que se lo de o su tiempoEspera acabe.
CU-02 Preparación	El barista empieza a hacer cafés y notifica al camarero cuando ya esté listo.
CU-03 Coger el café cuando este listo	El camarero cuando le avisa el barista que el café esta listo lo recoge para entregarlo.
CU-04 Entrega	Si el cliente no se le termino el tiempo de espera, el camarero le entrega el café y se muestra el tiempo total de preparación de ese café por parte del camarero.
CU-05 Abandono	Si el tiempo de espera termina antes de la entrega, el cliente se marchará y el camarero pasa al siguiente cliente.

1.5. Historias de usuario

ID	Como...	Quiero...	Para...
HU-01	Cliente	Pedir mi café y esperar un tiempo límite	No perder el tiempo si no me atienden me marchó.
HU-02	Camarero	Atender al cliente que se le asigno	Atender por orden de llegada y entregar su café.
HU-03	Barista	Preparar los cafés si no hay ninguno en la lista	Cuando no haya cafés, los hará y notificara al camarero para para que los pueda entregar si hay clientes.
HU-04	Usuario	Poder ver el estado del programa	Saber que clientes están atendidos y por quien y saber quien recibe su café o se marcha.

2. Arquitectura del sistema

2.1. Arquitectura

La arquitectura de esta aplicación se explicará en base a lo realizado en **JavaFX** ya que sigue una arquitectura **Modelo–Vista–Controlador (MVC)** sencillo que corre únicamente en local.

- **Modelo:** cuenta con las clases **cliente**, **camarero**, **barista**, **buffer** con la lógica, el barista prepara los cafés, los camareros los reparten cuando están listos al cliente que están atendiendo y notifican cuando terminan y el cliente espera su café hasta que se lo sirvan o se agote su tiempo de espera y se marche.
- **Vista:** muestra los **clientes** y **camareros** con sus estados si están esperando, atendidos, si les sirvieron o si se fueron y si están libres o atendiendo un cliente y también los **cafés preparados** por el barista que están en la lista del buffer a la espera de que llegue un cliente y el camarero se lo entregue.
- **Controlador:** Responde al botón de iniciar la simulación y actualiza los contenedores para mostrar el estado de los clientes, camareros y cafés preparados en la interfaz.

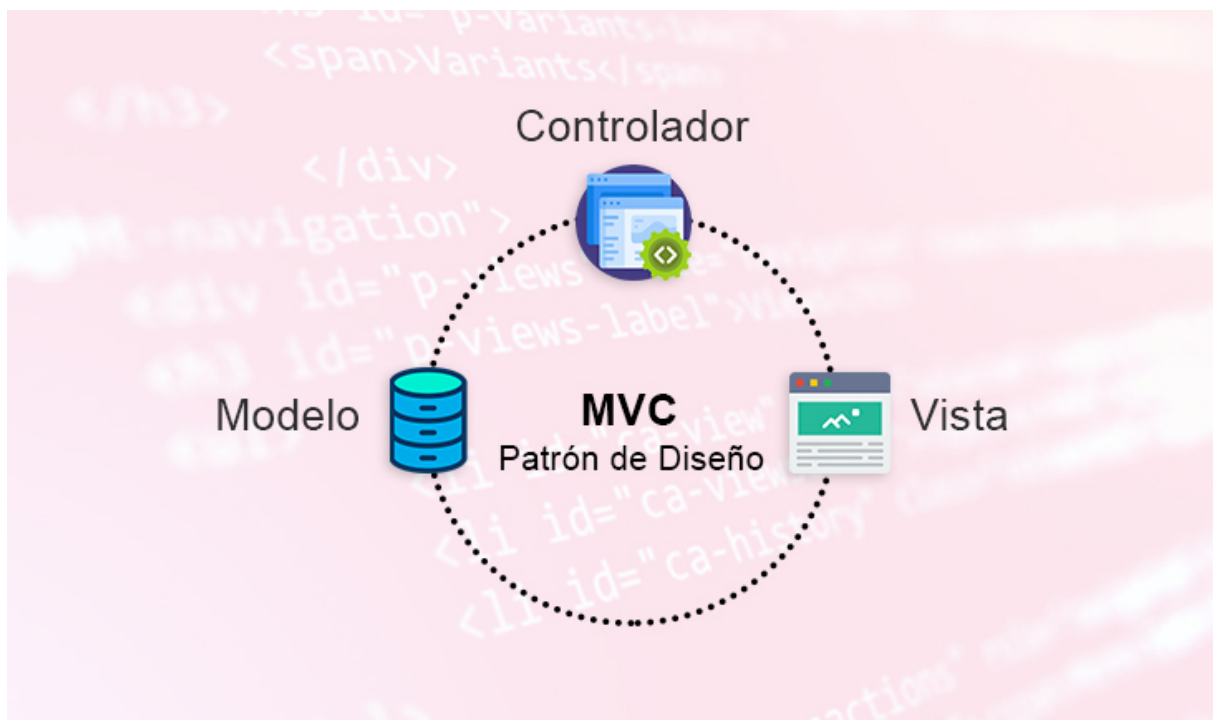


Figura 1: Arquitectura MVC

2.2. Clases

En este caso tenemos 4 clases principales la de **Clientes**, **Camareros**, **Barista** y la de **Buffer**:

Cliente/Client

Client representa a un cliente que entra en la cafetería, pide su café y espera un tiempo máximo (`waitingTime`) antes de marcharse si no lo recibe.

Atributos:

- **Identificación:** `name` (nombre del cliente), `waitingTime` (tiempo que está dispuesto a esperar en ms).
- **Estados:** `inFile` (si está en la cola), `assigned` (si está atendido), `served` (si ya le sirvieron el café), `left` (si ya se marchó).
- **Tiempos:** `arriveAt` (instante en el que llega), `timeStart` (inicio del tiempo de espera), `preparationTime` (cuanto tardo el camarero en hacer su café).

Flujo: Al inicio, el cliente marca su llegada (`timeStart`, `arriveAt`, `inFile = true`) y entra en un bucle esperando a que `served` pase a `true`. Mientras tanto, se comprueba si el tiempo de espera actual supera a `waitingTime`. Si esto pasa:

- La variable para indicar que se marchó pasa a `left = true`.
- El hilo del cliente termina, ya que se marchó de la cafetería.

Si el camarero le entregó el café a tiempo (`served = true` antes de que se agote `waitingTime`), el cliente termina y muestra el mensaje con el `preparationTime`.

Camarero/Waiter

Waiter representa a un camarero que atiende solamente a un cliente asignado cada vez, entregándole el café al cliente cuando esté ya esté preparado.

Atributos:

- **Identificación:** `name` (nombre del camarero).
- **Control:** `assignment` (cliente asignado actual), `running` (permite poder parar el bucle).
- **Buffer:** `buffer` para que el camarero tome los cafés preparados.
- **Temporización:** `timeStart` (para medir la duración desde que empieza con la preparación).

Flujo: Al inicio, el camarero entra en un bucle continuo mientras `running` sea `true` o tenga un `assignment` pendiente:

- Si no tiene un cliente asignado (`assignment == null`):
 - Sigue esperando un poco (`Thread.sleep(15)`).
 - Comprueba si el `CoffeController` ya le ha asignado un cliente.
- Si tiene un cliente asignado que se ha marchado (`client.left`):
 - Se libera la asignación (`assignment = null`) y queda disponible para atender a otro cliente.
- Si tiene un cliente asignado que sigue esperando:
 - Registra el `timeStart`.
 - Obtiene un café del `Buffer` mediante `buffer.takeCoffe()`.
 - Calcula el `preparationTime` para saber cuanto tardo en preparar/entregar el café.

Tras obtener el café preparado por el barista, hará lo siguiente:

- Si el cliente no se marchó:
 - La variable para indicar que está servido pasa a `client.served = true`.
 - Y le pasa el `preparationTime` al cliente para mostrar cuánto tardó en recibir su café.
- Si el cliente se marchó mientras esperaba el café:
 - Se muestra un mensaje que informa que el cliente se marchó sin su café.
- Al finalizar:
 - El camarero se liberará `assignment = null` para poder atender a otro cliente.

El método `setBuffer(Buffer buffer)` nos permite actualizar el `Buffer` por si agregamos un nuevo camarero para que el barista haga un café más.

Barista

Barista representa al trabajador encargado de preparar los cafés de forma continua.

Atributos:

- **Buffer:** `buffer` (referencia al objeto `Buffer` donde se almacenan los cafés preparados).
- **Control de ejecución:** `running` indica si el barista debe seguir trabajando o puede detenerse.
- **Simulación:** `random` (instancia de `Random`) para generar tiempos de preparación variables y hacer la simulación más realista.

Flujo: Al inicio, el barista entra en un bucle continuo mientras `running` sea `true`:

- Simula el tiempo de preparación del café con `Thread.sleep()` en el método `prepareCoffe()`.
- Añade el café que preparo al buffer mediante `buffer.addCoffe(coffe)`.
- Si el `Buffer` está lleno, el barista se bloquea hasta que se libere.

Buffer

Buffer es donde se guardan los cafés ya preparados por el barista y que los camareros recogerán.

Atributos:

- **Cola de cafés:** `coffesList` lista que almacena los cafés preparados.
- **Capacidad:** `capacity` indica el número máximo de cafés que puede haber preparados a la vez. Se calcula segun el número de camareros que haya.

Métodos principales:

- `setCapacity(int newCapacity)`: para poder actualizar la capacidad del buffer. Llama a `notifyAll()` para avisar a los hilos que puedan estar esperando por cafés.
- `addCoffe(String coffe)`: método que usa el **Barista** para añadir un café al buffer.
 - Si la cola está llena (`coffesList.size() == capacity`), bloqueamos al barista con `wait()` hasta que un camarero consuma un café.
 - Cuando añade el café, se llama a `notifyAll()` para avisar a los camareros que estén esperando cafés.
- `takeCoffe()`: método que usan los **Camareros** para recoger un café.
 - Si la cola está vacía, bloqueamos al camarero con `wait()` hasta que el barista prepare un nuevo café.
 - Cuando hay cafés disponibles, se sacamos el primero de la lista.
 - Tras recoger un café se llama a `notifyAll()` para notificar al barista de que vuelve a haber espacio y que preparé un café.
- `getCoffesList()`: devuelve una copia de la lista de cafés actuales en el buffer (para mostrar estado de la lista de los cafés en la interfaz).

2.3. Patrones de diseño

Ahora vamos a explicar el **patrón de diseño empleado**, que nos permiten estructurar el flujo de ejecución y como interactúan. Usar los patrones nos ayuda a mantener el **código modular, legible y ampliarlo fácilmente** en un futuro.

- Patrón Productor-Consumidor

- **Productor**: genera elementos y los coloca en un *buffer* compartido.
- **Consumidores**: sacan los elementos del buffer cuando lo necesiten.
- **Buffer**: lista que almacena los elementos.

En nuestra aplicación lo aplicamos de la siguiente manera:

- El **Barista** es el **productor**, que prepara los cafés y los añade al **Buffer** cuando tenga espacio disponible.
- Los **Camareros** son los **consumidores**, sacando los cafés del **Buffer** para entregarlos a los clientes que están atendiendo.
- El **Buffer** es la lista compartida con los cafés preparados.

La sincronización entre hilos se realiza con los métodos `wait()` y `notifyAll()`:

- **Buffer completo**, el **Barista** se bloquea hasta que algún camarero saque cafés de la lista.
- **Buffer vacío**, los **Camareros** se bloquean hasta que el barista prepare un nuevo café.

Esto nos garantiza:

- No se preparen más cafés de los que los camareros pueden gestionar (control de capacidad).
- Los camareros no intenten entregar un café que todavía no existe.

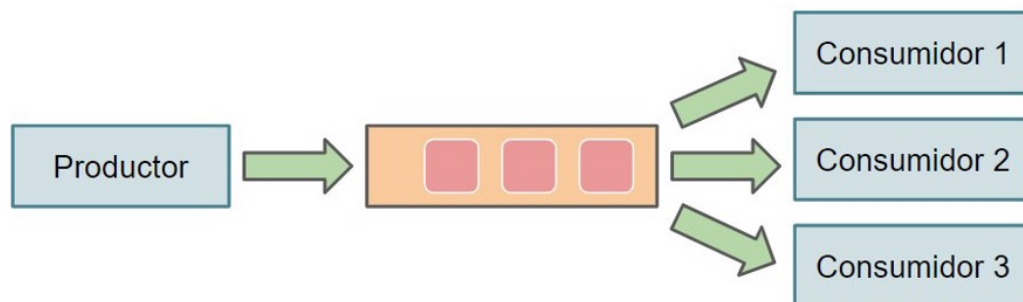


Figura 2: Patrón productor/consumidor

2.4. Stack empleado

Para el desarrollo nuestra aplicación tanto en terminal como con interfaz, se utilizaron las siguientes tecnologías:

- Utilizamos Java como **entorno de ejecución** principal.
- Utilizamos JavaFX para crear una **interfaz con Java**.

2.5. Interfaz

Se creo una interfaz muy simple en JavaFX, para **mostrar el programa de forma clara y bonita**, para saber cuando son atendidos los clientes y cuanto tardan en recibir su café o si se han ido. Como podemos apreciar tenemos un **título** y debajo el **botón** para comenzar con esta simulación del servicio de la cafetería y luego tenemos **tres contenedores** uno para clientes, uno para los camareros donde mostramos su estado y un contenedor para los cafés preparados por el barista que están en cola de espera.

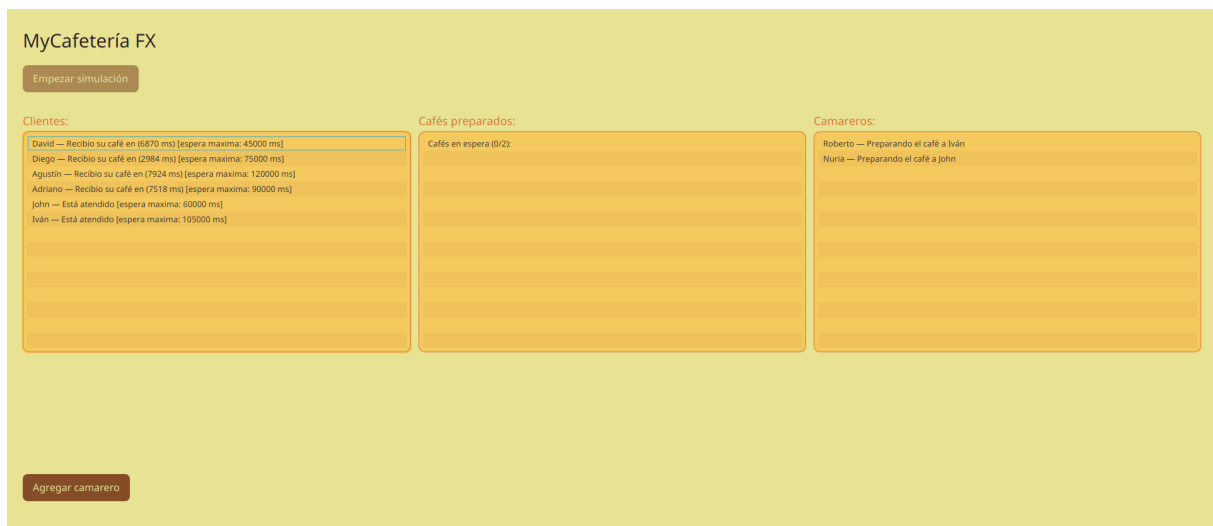


Figura 3: Interfaz en JavaFX